

Classification

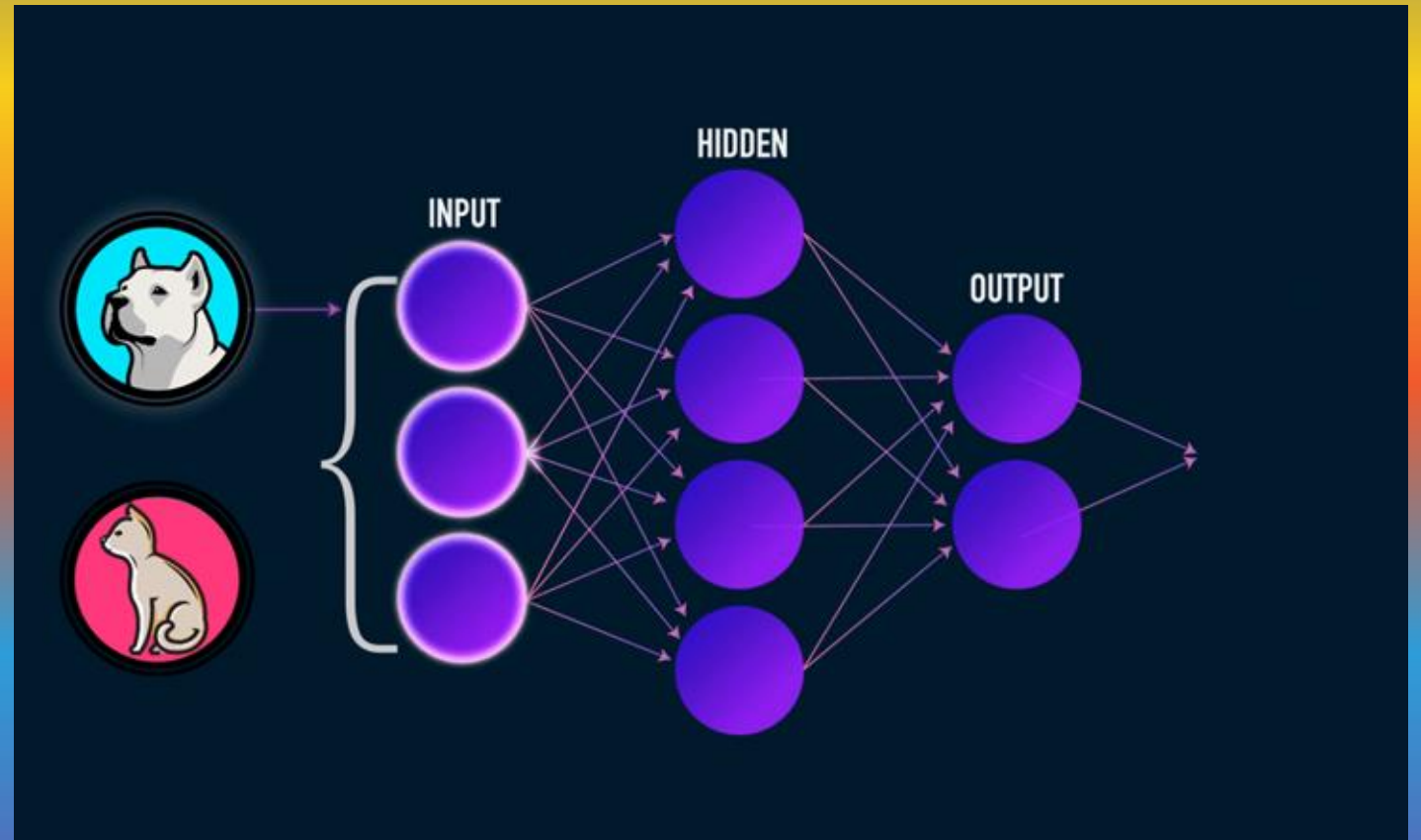
Distance & Feature Space

KNN & Confusion Matrix

Solar Flare Classification (Jupyter Notebook)

What is Classification?

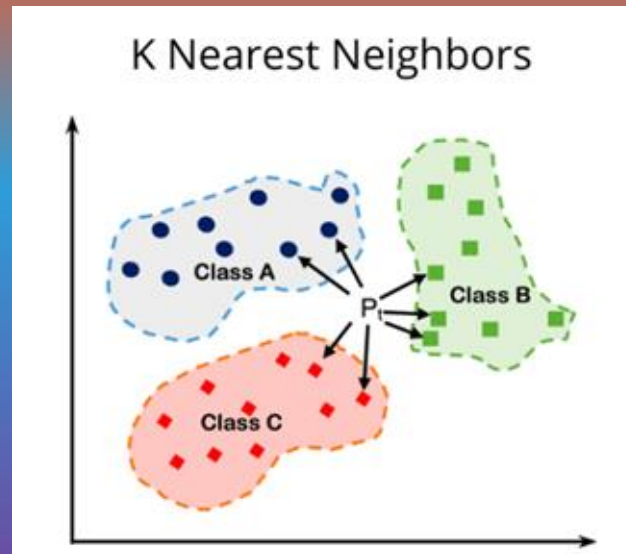
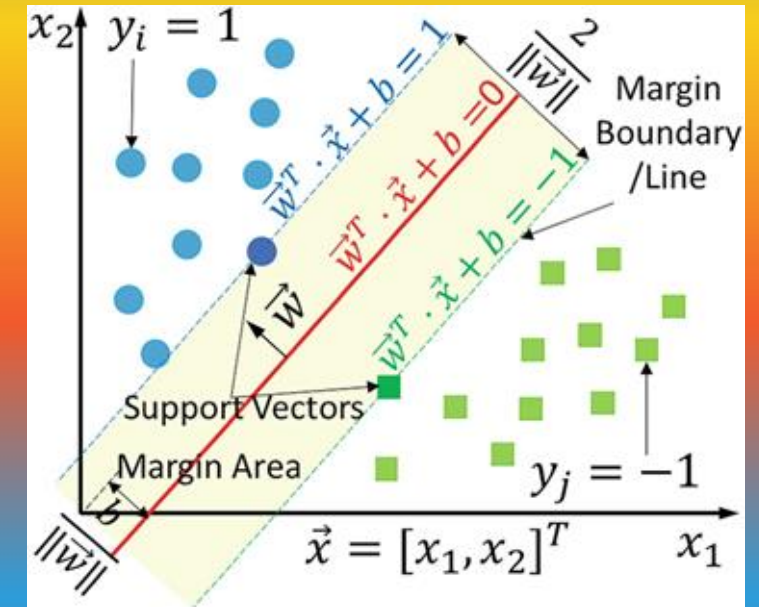
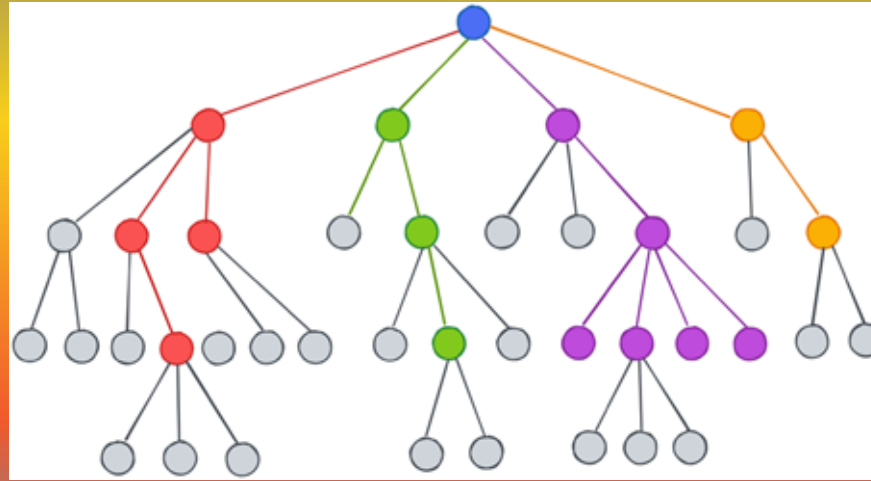
https://youtu.be/QM0sYbEQSkM?si=6Jptf4jIV6-iP_GM



Types of Classification Models

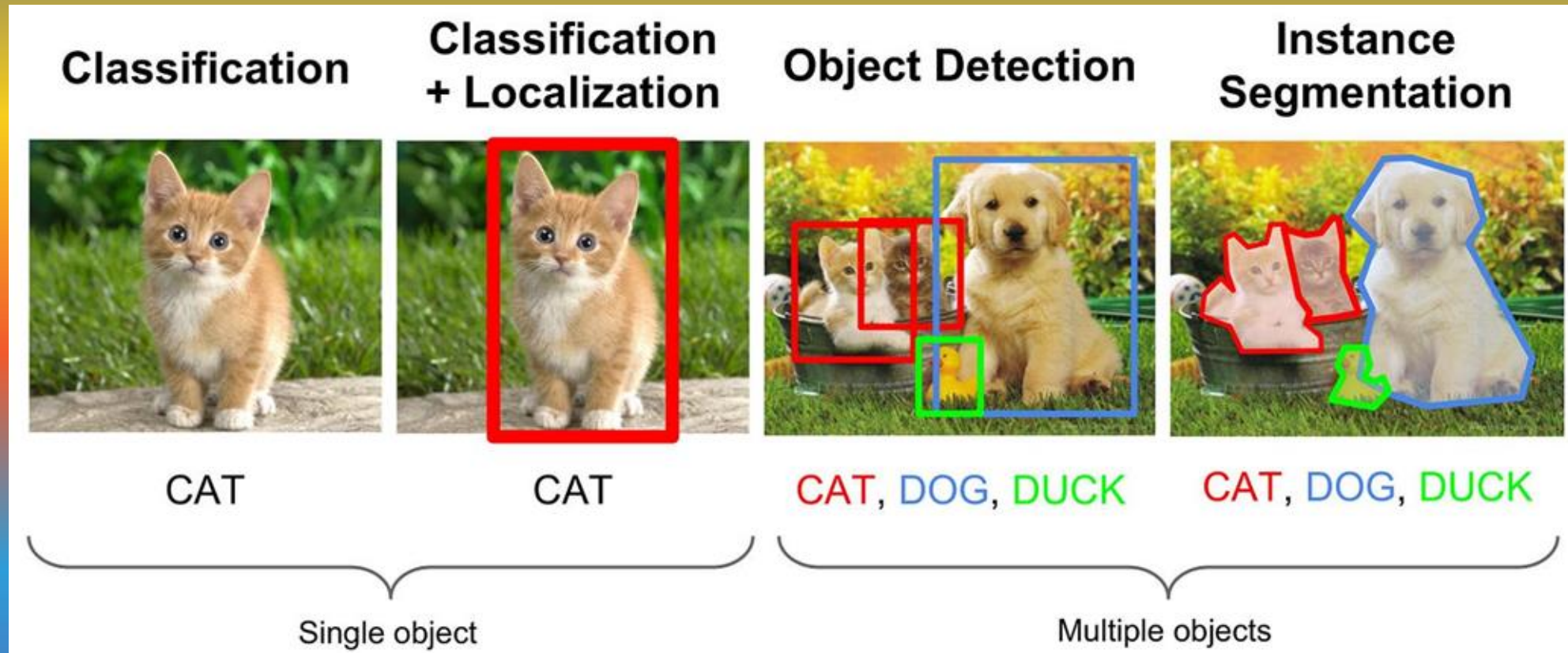
- Types:

- Decision Trees
- Support Vector Machines
- K-Nearest Neighbors



<https://youtu.be/e-IHu0U9UY4?si=0rmDzjEJyJEqv78>
0

Classification Types



<https://youtu.be/wVE8SFMSBJ0?si=3mPijtK0n9ukGaHl>

<https://youtu.be/3BUTirVsivc?si=LEwunBchKFe64EUN>

Feature Space

A feature is a measurable property that describes an object.

Object Being Classified	Features	Possible Labels
Star	Temperature, luminosity, radius, mass	Main Sequence, Giant, White Dwarf
Solar Image	Mean intensity, standard deviation, entropy, maximum brightness, bright pixel fraction	Flare, Non-flare
Person	Height, weight, speed, endurance	Basketball Player, Dancer, Runner, Swimmer

Feature Space



(mean brightness, radius/size) $\rightarrow$ (300, 92)
label: spiral



(mean brightness, radius/size) $\rightarrow$ (120, 18)
label: elliptical

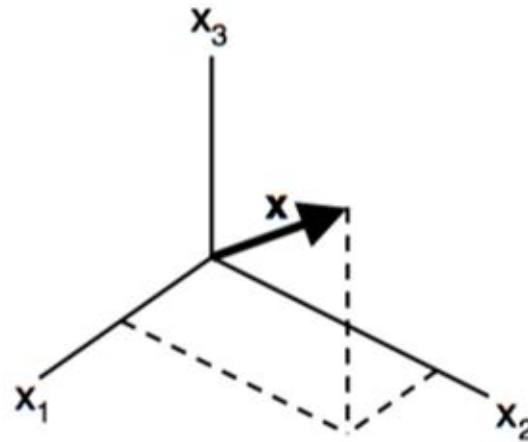


(mean brightness, radius/size) $\rightarrow$ (124, 19)
label?

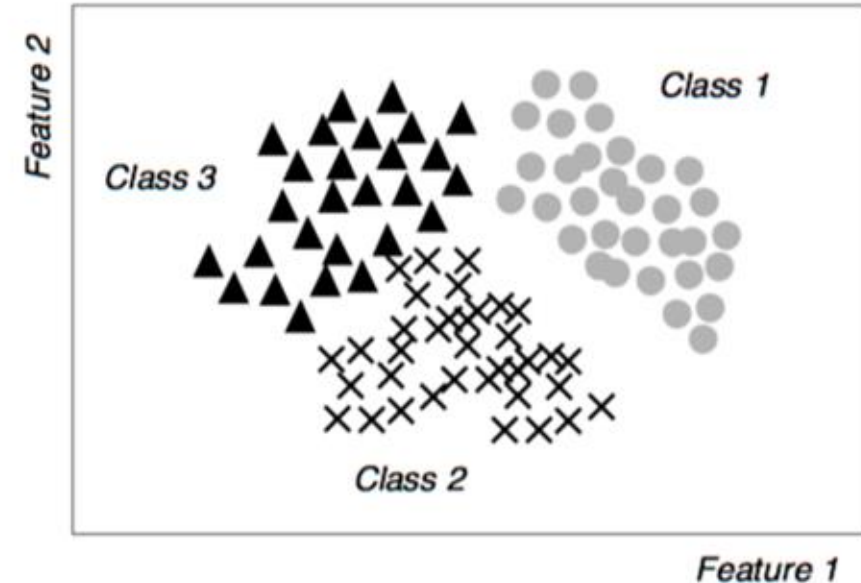
Feature Space

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

Feature vector



Feature space (3D)



Scatter plot (2D)

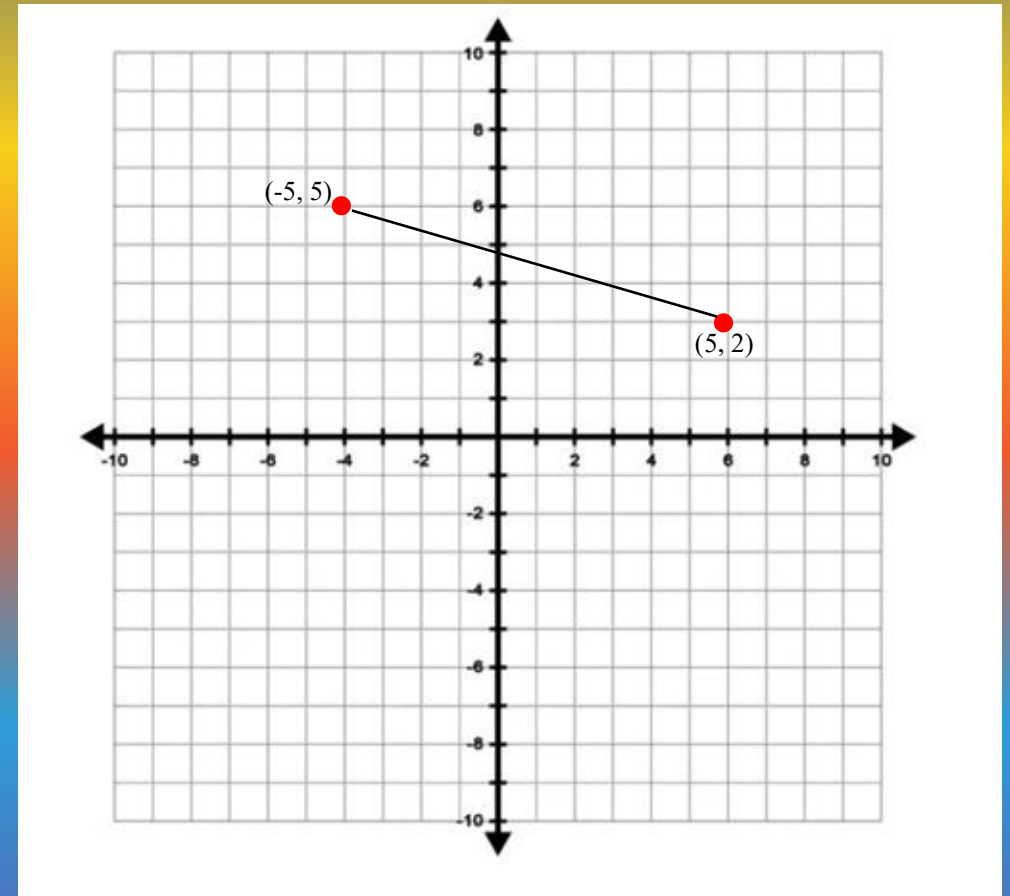
A feature vector is the list of numbers that describes one object. For example, one solar image could be represented as [mean intensity, standard deviation, entropy].

Feature space is where the feature vector "lives." If an object has three features, we can plot it in 3D as $((x_1, x_2, x_3))$.

A scatter plot is a 2D view of feature space. Each point is one object, and points from the same class often cluster together.

Distance: Euclidean

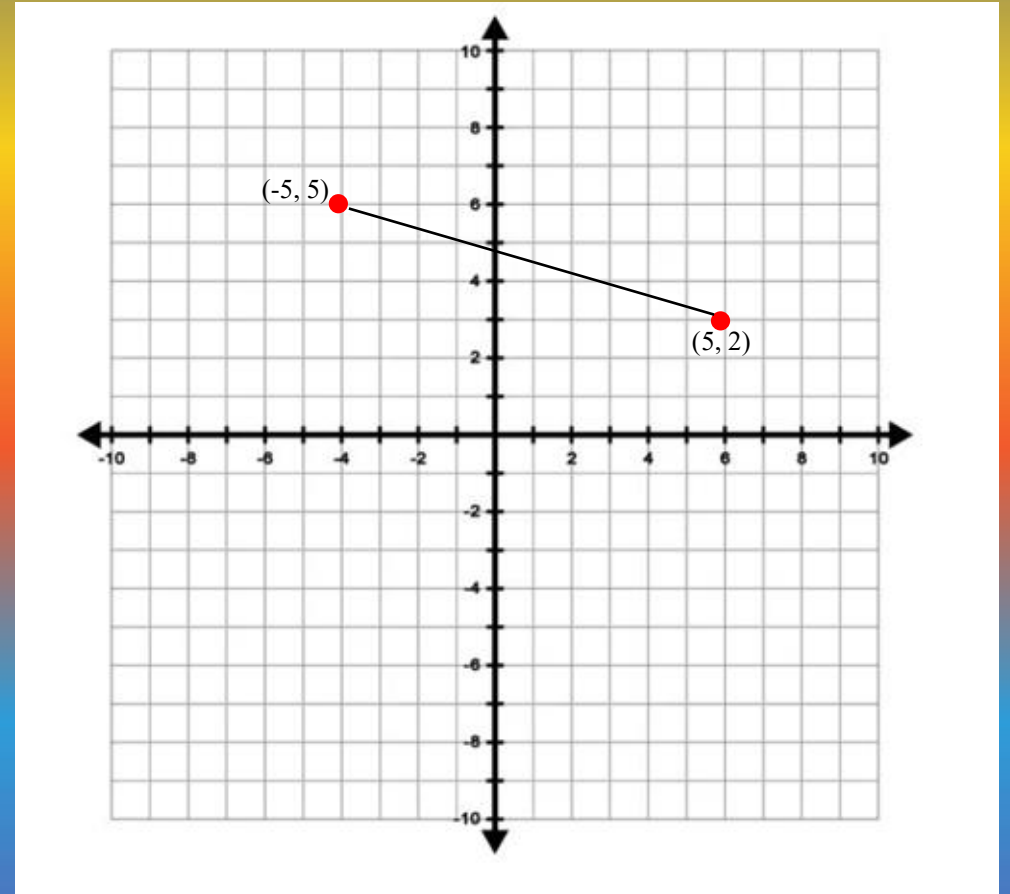
$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



matches our idea of visual distance

Distance: Manhattan

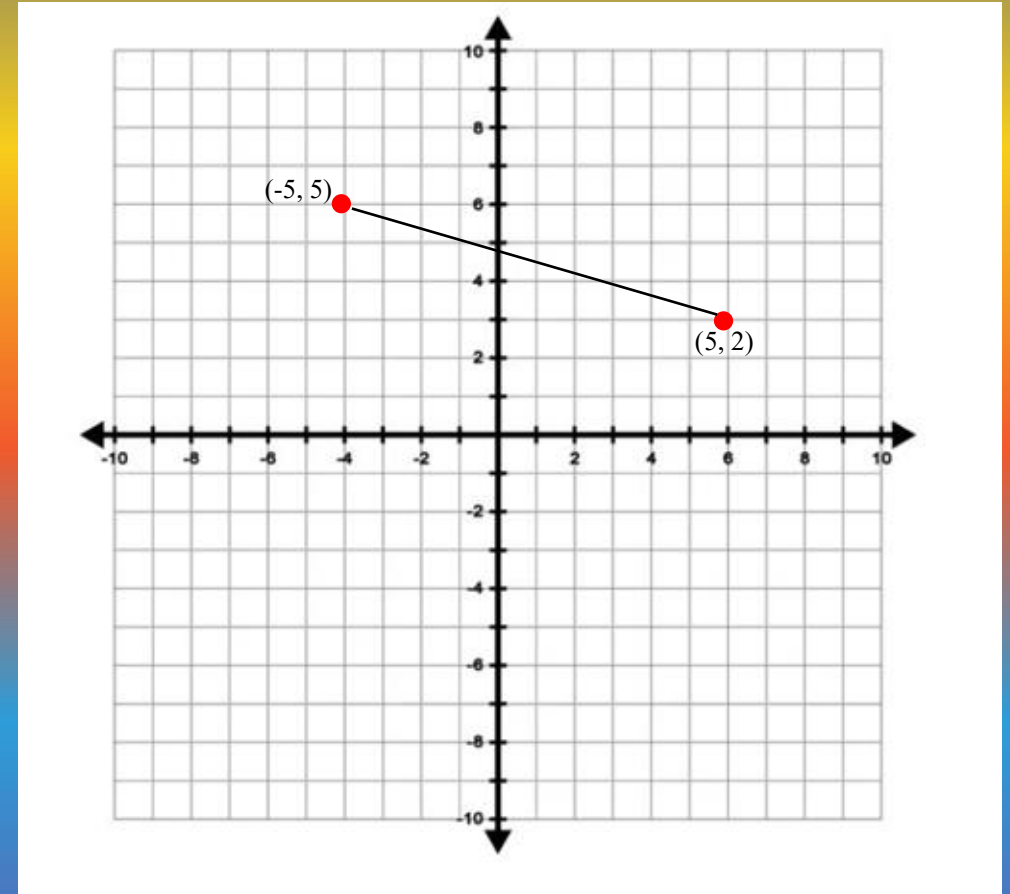
$$d = |x_2 - x_1| + |y_2 - y_1|$$



longer because it forces you to move across and down

Distance: Minowski

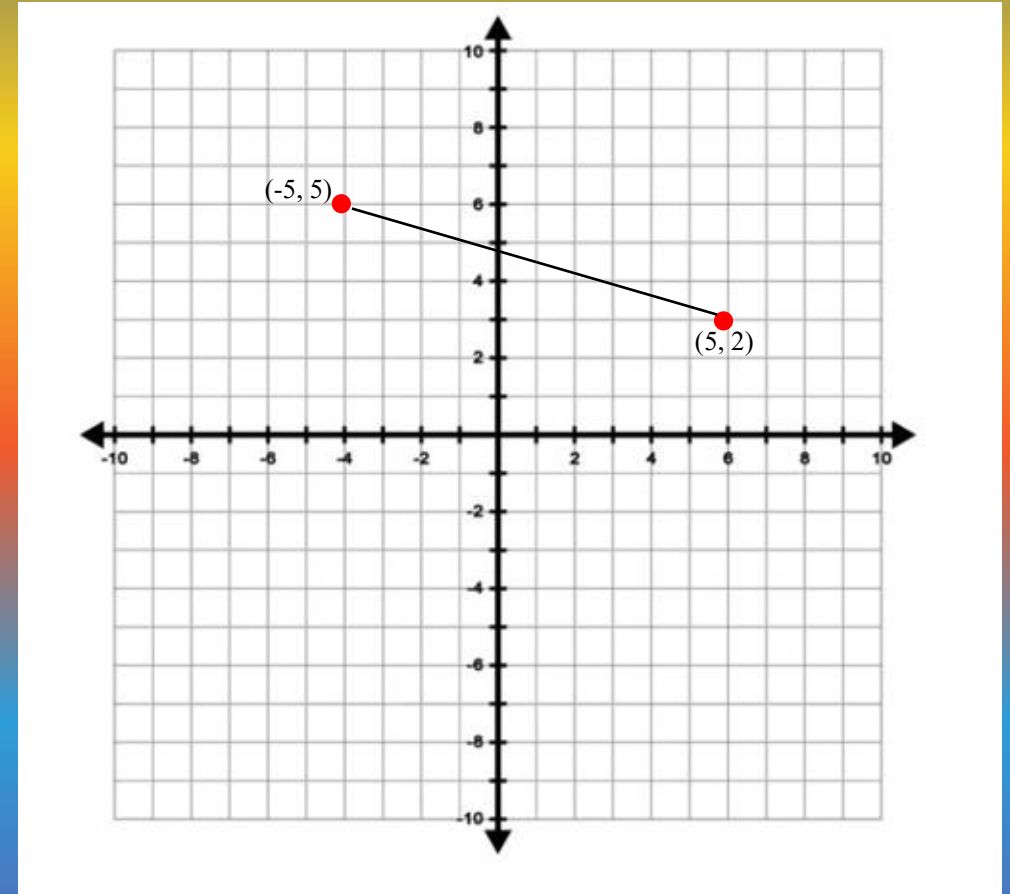
$$d = (|x_2 - x_1|^p + |y_2 - y_1|^p)^{1/p}$$



depends on how you choose p ,
if $p = 1$, manhattan, if $p = 2$, euclidean

Distance: Chebyshev

$$d = \max(|x_2 - x_1|, |y_2 - y_1|)$$



shorter because it only cares about shortest distance

K Nearest Neighbors



<https://youtu.be/0p0o5cmgLdE?si=oVLFxsqnbYTztkOF>

KNN by Hand

Algorithm:

- Identify the feature vector?
- Choose k
- Compute distance of all points from new point to every training point
- Rank distances from smallest to largest
- Select k nearest neighbors
- Categorized based on k

K Nearest Neighbors

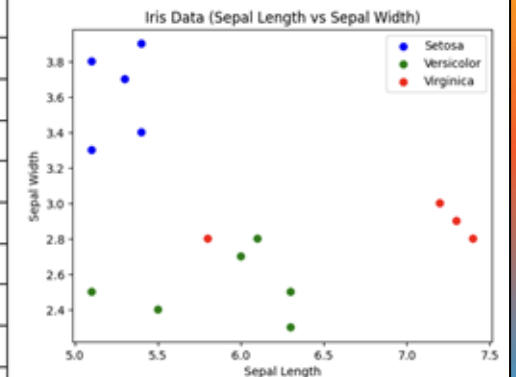
Given the following sepal data, classify its species: Length = 5.2, Width = 3.1

Step 1) Find the Euclidean distance from the test point to each point in the sample

Step 2) Rank each point from smallest to largest

Step 3) Use the given k to find the species based on the majority

Length	Width	Species	$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$	Rank	k = 1	k = 3	k = 5
5.3	3.7	Setosa					
5.1	3.8	Setosa					
7.2	3.0	Virginica					
5.4	3.4	Setosa					
5.1	3.3	Setosa					
5.4	3.9	Setosa					
7.4	2.8	Virginica					
6.1	2.8	Versicolor					
7.3	2.9	Virginica					
6.0	2.7	Versicolor					
5.8	2.8	Virginica					
6.3	2.3	Versicolor					
5.1	2.5	Versicolor					
6.3	2.5	Versicolor					
5.5	2.4	Versicolor					



This table is for making one KNN prediction by hand.

To calculate accuracy, we need several test points with known true labels.

Solar Flare Example

In this ML exercise, we will test the model on many solar cutouts at once and calculate how many predictions were correct.

Click Here: <http://149.165.147.51/hub/login>

- Solar flare images
- Feature extraction
- Standardization
- KNN implementation
- Confusion matrix

		Ground truth		
		+	-	
Predicted	+	True positive (TP)	False positive (FP)	Precision = TP / (TP + FP)
	-	False negative (FN)	True negative (TN)	
		Recall = TP / (TP + FN)		Accuracy = (TP + TN) / (TP + FP + TN + FN)

How to optimise trading strategies using KNN?



Guided Example: K-Nearest Neighbors (KNN) for Solar Flare Image Classification

This notebook demonstrates a simple end-to-end KNN classification workflow:

1. Solar flare images (128x128 cutouts)
2. Feature extraction (turn each image into a feature vector)
3. Standardization (feature scaling)
4. KNN implementation
5. Confusion matrix interpretation

Big idea

KNN does not "learn" parameters like linear regression.

Instead, it stores labeled training points in **feature space** and classifies new samples by looking at the **k nearest** training points.

```
# If needed, install dependencies (uncomment)
# !pip3 install numpy pandas matplotlib scikit-learn pillow requests

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, accuracy_score, classification_report

plt.rcParams['figure.figsize'] = (7, 4)
```

1) Load images from your S3 bucket

You said these will be pulled from S3. The cleanest approach is a **manifest CSV** with columns like:

- `image_url`
- `label` (0 = non-flare, 1 = flare)
- `event_id` (used to split train/test by event)

Replace `MANIFEST_URL` with your real S3 link when ready.

```
MANIFEST_URL = None # e.g., "https://.../flare_cutouts_manifest.csv"

# Fallback: tiny example list (replace with your actual URLs if not using a manifest)
example_rows = [
    # ("image_url": "https://.../flare_001.png", "label": 1, "event_id": "E001"),
    # ("image_url": "https://.../nonflare_002.png", "label": 0, "event_id": "N002"),
]

if MANIFEST_URL:
    df = pd.read_csv(MANIFEST_URL)
else:
    df = pd.DataFrame(example_rows)

df.head(), df.shape
```

Helper: Download + load image into a 2D array

- Converts to grayscale
- Resizes to 128x128 (if needed)

```
import requests
from PIL import Image
from io import BytesIO

def load_grayscale_image_from_url(url, size=(128, 128)):
    r = requests.get(url, timeout=30)
    r.raise_for_status()
    img = Image.open(BytesIO(r.content)).convert("L") # grayscale
    if size is not None:
        img = img.resize(size)
    return np.asarray(img, dtype=float)
```

2) Feature extraction (per image)

Each image becomes a **feature vector**:

- Mean intensity
- Standard deviation (contrast)
- Maximum intensity
- Fraction of pixels above a threshold